Guion: Práctica 8
Presentador: Bienvenidos a la explicación de esta práctica. Actualmente los estudiantes usan los modelos de lenguaje a gran escala para consultar conceptos técnicos específicos de programación e incluso conceptos aplicados de otras disciplinas, lo que genera un problema ya que muchos usan la información sin verificarla y sin saber si es correcta. Al mismo tiempo, al programar en Java, el uso de interfaces y polimorfismo para modelar clases distintas que comparten un comportamiento común representa un reto para el estudiantado, pues requiere comprender cómo diferentes clases pueden implementar un mismo contrato, definido por la interfaz, de forma independiente y sin duplicar código. Por eso los objetivos fueron: elaborar un programa donde las clases Edificio, Auto y Bicicleta implementen la interfaz HuellaCarbono, aplicando el uso correcto de interfaces y polimorfismo; y aplicar la sobreescritura de métodos mediante @Override, para que cada clase calcule su huella de carbono y muestre su información con la fórmula y el formato correspondiente.

Orador 1: Del marco teórico, vimos la herencia, que permite que una clase hija adquiera las propiedades y los métodos de su clase padre, además de poder agregar o modificar su propio comportamiento. Por otro lado, el polimorfismo permite que un mismo método tenga diferentes comportamientos dependiendo de la clase del objeto que lo utiliza.

También vimos que `extends` establece la herencia entre clases; `instanceof` comprueba si un objeto es instancia de una clase; `toString` permite representar la información de un objeto; y `@Override` indica que se está sobrescribiendo un método heredado. Además, `super` permite acceder a elementos de la clase padre.

Por último, vimos la sobrecarga de métodos, que permite utilizar el mismo nombre para métodos con diferentes parámetros, y la sobrecarga de constructores, que permite inicializar objetos de una misma clase de distintas maneras.

Orador 2: Además, usamos el concepto de interfaces. Una interfaz es una clase abstracta pura, es decir, donde todos sus métodos son abstractos y no se implementa ninguno; no se pueden crear instancias de una interfaz. Permite establecer la forma de una clase, con nombres de métodos, listas de argumentos y tipos de retorno, pero no bloques de código. Para declararla se usa la palabra interface en lugar de class, y todos sus métodos son públicos y abstractos, mientras que sus atributos, si los tiene, son públicos, estáticos y finales. En el desarrollo, primero se le preguntó a un LLM cómo calcular la huella de carbono de un edificio, un auto y una bicicleta. La fórmula general que arrojó fue dato de actividad por factor de emisión: para el edificio, el consumo en kilowatts-hora por 0.50 más el consumo de gas en metros cúbicos por 2.00; para el auto, los litros consumidos, calculados dividiendo los kilómetros recorridos entre el rendimiento, multiplicados por 2.31; y para la bicicleta, los kilómetros recorridos por 0.005, como aproximación del ciclo de vida de su fabricación y mantenimiento.

Presentador: Con esas fórmulas, se creó primero la interfaz HuellaCarbono, estableciendo que cada clase que la implemente debe incluir el método getHuellaCarbono, que retorna un dato de tipo double.

Orador 3: La clase Bicicleta implementa HuellaCarbono, con los atributos marca, numeroVelocidades y kilometrosPedaleados, todos privados, con su constructor y sus getters. Se programó el método cambiarVelocidad, que valida con if-else que la velocidad exista en el modelo; con override se sobreescribió getHuellaCarbono, multiplicando los kilómetros pedaleados por 0.005, y también toString, para mostrar la información de la bicicleta. La clase Auto, que también implementa HuellaCarbono, tiene como atributos modelo, kilometrosRecorridos y rendimientoKml, privados, con su constructor y getters. Se programó getLitrosConsumidos, que valida con if-else que el rendimiento sea mayor a cero antes de dividir los kilómetros entre él; con override se sobreescribió getHuellaCarbono, multiplicando los litros consumidos por 2.31, y también toString.

Orador 4: La clase Edificio implementa HuellaCarbono, con los atributos nombre, consumoElectricoKwh y numeroPisos, privados, con su constructor y getters. Se programó encenderLucesPiso, que valida con if-else que el piso exista en el edificio; con override se sobreescribió getHuellaCarbono, multiplicando el consumo eléctrico por 0.438, y también toString. En la clase Main se instanció un objeto de cada clase, se invocaron sus métodos propios, y se guardaron en un ArrayList para recorrerlo e imprimir la huella de carbono de cada uno. En los resultados, cada clase mostró su comportamiento exclusivo: el Edificio indicó cuántas luces están encendidas en el piso especificado, o avisó si el piso no existe; el Auto hizo sonar su claxon cada vez que se le indicó; y la Bicicleta estableció su número de velocidades y validó los cambios dentro de ese rango. Cada objeto, al recorrer la lista, calculó correctamente su huella de carbono con la fórmula propia de su clase.

Presentador: Para cerrar, se cumplieron los objetivos de la práctica al implementar la interfaz HuellaCarbono en las clases Edificio, Auto y Bicicleta. El uso del polimorfismo y @Override demostró cómo cada objeto puede calcular sus propias emisiones de forma independiente, evitando la duplicación de código. Y se usaron correctamente los modelos de lenguaje para validar las fórmulas ambientales y resolver el ejercicio de manera satisfactoria. Gracias por acompañarnos en esta explicación.